1: BERT Sentiment Classifier on Movie Reviews

Loading the datset

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
!pip install transformers
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.57.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.20.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.36
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0->t
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.5.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2025
```

```
import pandas as pd
import numpy as np

df=pd.read_csv("/content/drive/MyDrive/datasets/bert_c/IMDB_Dataset.csv")
print(df)
df.info()
```

```
                                                  review sentiment
0      One of the other reviewers has mentioned that ...  positive
1      A wonderful little production. <br /><br />The...  positive
2      I thought this was a wonderful way to spend ti...  positive
3      Basically there's a family where a little boy ...  negative
4      Petter Mattei's "Love in the Time of Money" is...  positive
...                                                  ...       ...
49995  I thought this movie did a down right good job...  positive
49996  Bad plot, bad dialogue, bad acting, idiotic di...  negative
49997  I am a Catholic taught in parochial elementary...  negative
49998  I'm going to have to disagree with the previou...  negative
49999  No one expects the Star Trek movies to be high...  negative

[50000 rows x 2 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   review     50000 non-null  object
 1   sentiment  50000 non-null  object
dtypes: object(2)
memory usage: 781.4+ KB
```

Cleaning and Preprocessing

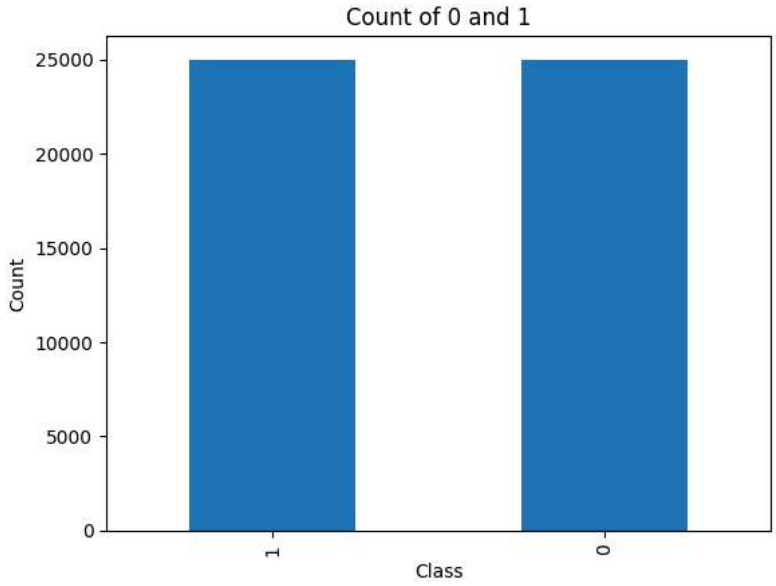
```
sd=df.groupby('sentiment').size()
print(sd)
sd.shape
```

```
sentiment
negative    25000
positive    25000
dtype: int64
(2,)
```

```
df['sentiment'] = df['sentiment'].replace({
    'positive':1,
    'negative': 0})
print(df.info())
print(df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype 
---  ------     --------------  ----- 
 0   review     50000 non-null  object
 1   sentiment  50000 non-null  int64 
dtypes: int64(1), object(1)
memory usage: 781.4+ KB
None
          sentiment
count  50000.000000
mean       0.500000
std        0.500005
min        0.000000
25%        0.000000
50%        0.500000
75%        1.000000
max        1.000000
/tmp/ipython-input-915099207.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a futur
  df['sentiment'] = df['sentiment'].replace({
```

```
import matplotlib.pyplot as plt
df['sentiment'].value_counts().plot(kind='bar')
plt.xlabel('Class')
plt.ylabel('Count')
plt.title('Count of 0 and 1')
plt.show()
```



Tokenizing

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%                                                 48.0/48.0 [00:00<00:00, 1.21kB/s]

vocab.txt: 100%                                                          232k/232k [00:00<00:00, 3.51MB/s]

tokenizer.json: 100%                                                        466k/466k [00:00<00:00, 11.2MB/s]

config.json: 100%                                                         570/570 [00:00<00:00, 11.1kB/s]
```

```
def bert_tokenize(texts, max_len=128):
    return tokenizer(
        texts.tolist(),
        padding='max_length',
        truncation=True,
        max_length=max_len,
        return_tensors='pt'
    )

encodings = bert_tokenize(df['review'])
```

```
print(encodings.keys())
```

```
KeysView({'input_ids': tensor([[  101,  2028,  1997,  ...,  2004,  2008,   102],
        [  101,  1037,  6919,  ...,  2537,  2055,   102],
        [  101,  1045,  2245,  ..., 22092,  1999,   102],
        ...,
        [  101,  1045,  2572,  ...,  2058,  3772,   102],
        [  101,  1045,  1005,  ...,  2112,  1997,   102],
        [  101,  2053,  2028,  ...,  1010,  2174,   102]]), 'token_type_ids': tensor([[0, 0, 0,  ..., 0, 0, 0],
        [0, 0, 0,  ..., 0, 0, 0],
        [0, 0, 0,  ..., 0, 0, 0],
        ...,
        [0, 0, 0,  ..., 0, 0, 0],
        [0, 0, 0,  ..., 0, 0, 0],
        [0, 0, 0,  ..., 0, 0, 0]]), 'attention_mask': tensor([[1, 1, 1,  ..., 1, 1, 1],
        [1, 1, 1,  ..., 1, 1, 1],
        [1, 1, 1,  ..., 1, 1, 1],
        ...,
        [1, 1, 1,  ..., 1, 1, 1],
        [1, 1, 1,  ..., 1, 1, 1],
        [1, 1, 1,  ..., 1, 1, 1]])})
```

```
import torch
labels = torch.tensor(df['sentiment'].values)
```

```
from torch.utils.data import Dataset

class SentimentDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: val[idx] for key, val in self.encodings.items()}
        item['labels'] = self.labels[idx]
        return item

    def __len__(self):
        return len(self.labels)

dataset = SentimentDataset(encodings, labels)
```

```
tokens = tokenizer.convert_ids_to_tokens(encodings['input_ids'][0])
print(tokens)
```

```
['[CLS]', 'one', 'of', 'the', 'other', 'reviewers', 'has', 'mentioned', 'that', 'after', 'watching', 'just', '1', 'oz', 'episode
```

```
from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
model.safetensors: 100%  440M/440M [00:03<00:00, 246MB/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

```
from sklearn.model_selection import train_test_split

train_idx, val_idx = train_test_split(
    range(len(dataset)),
    test_size=0.2,
    stratify=df['sentiment'],
    random_state=42
)

train_dataset = torch.utils.data.Subset(dataset, train_idx)
val_dataset = torch.utils.data.Subset(dataset, val_idx)
```

Biniary sentiment analyzer model

```
from transformers import Trainer, TrainingArguments

training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=5,
    fp16=True,
    weight_decay=0.01,
    logging_dir="./logs",
    load_best_model_at_end=True
)

import numpy as np
from sklearn.metrics import accuracy_score

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    return {
        "accuracy": accuracy_score(labels, preds)
    }
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    compute_metrics=compute_metrics
)

trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence '\/'
  | |_| | '_ \/ _` / _` |  _/ -_)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 3
wandb: You chose "Don't visualize my results"
Tracking run with wandb version 0.23.1
W&B syncing is set to `offline` in this directory. Run `wandb online` or set WANDB_MODE=online to enable cloud syncing.
Run data is saved locally in /content/wandb/offline-run-20251220_151922-lwredp17
```

[12500/12500 27:42, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Accuracy
1	0.281500	0.261419	0.893500
2	0.187900	0.271610	0.899100
3	0.103600	0.425265	0.897100
4	0.059900	0.534451	0.897200
5	0.025700	0.619967	0.897900

```
TrainOutput(global_step=12500, training_loss=0.14119561248779297, metrics={'train_runtime': 1671.6539,
'train_samples_per_second': 119.642, 'train_steps_per_second': 7.478, 'total_flos': 1.3155552768e+16, 'train_loss':
```

```
from sklearn.metrics import confusion_matrix,ConfusionMatrixDisplay

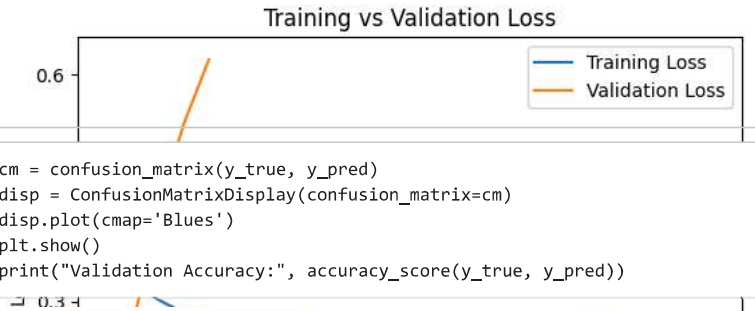
predictions = trainer.predict(val_dataset)
y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids
```

Final Evaluation

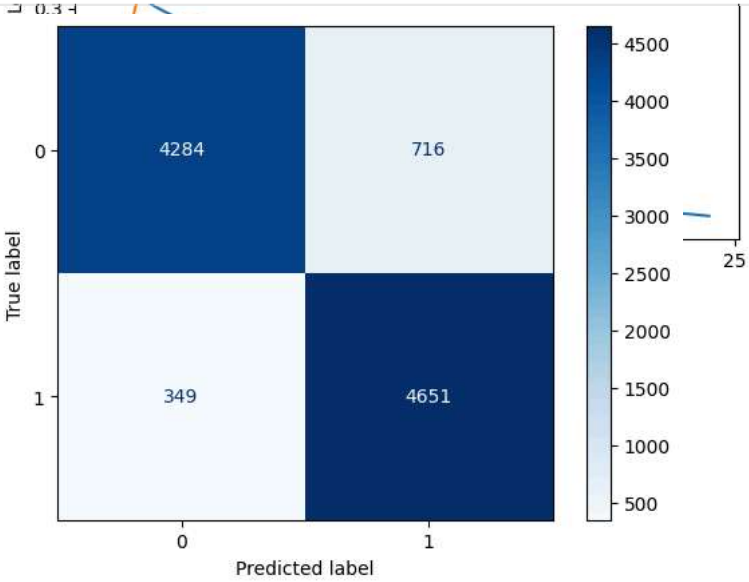
```
logs = trainer.state.log_history
train_loss = []
eval_loss = []
steps = []

for log in logs:
    if "loss" in log and "eval_loss" not in log:
        train_loss.append(log["loss"])
        steps.append(log.get("step", len(steps)))
    if "eval_loss" in log:
        eval_loss.append(log["eval_loss"])

plt.figure()
plt.plot(train_loss, label="Training Loss")
plt.plot(eval_loss, label="Validation Loss")
plt.xlabel("Evaluation Steps")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()
```




```
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.show()
print("Validation Accuracy:", accuracy_score(y_true, y_pred))
```




```
Validation Accuracy: 0.8935
```